

Potato Leaf Disease Detection

Training Code Explained in Simple Terms

A study guide for `train_model.py`

What is inside this guide

1. The project in one minute
2. Dataset and the 80/20 split
3. Libraries used
4. Code walk-through, section by section
5. How the CNN works, layer by layer
6. Reading the results
7. Limitations and future scope
8. Glossary
9. Questions an examiner may ask, with answers

1. The Project in One Minute

Problem: Potato farmers lose a lot of crop to leaf diseases. Two common ones are **Early Blight** and **Late Blight**. Spotting them early by eye needs an expert.

Our solution: We train a computer program (a deep learning model) to look at a photo of a potato leaf and say which of three classes it belongs to:

Class (folder name)	Meaning
Potato___Early_blight	Leaf infected with Early Blight (fungus <i>Alternaria solani</i>)
Potato___Late_blight	Leaf infected with Late Blight (<i>Phytophthora infestans</i>) - spreads very fast
Potato___healthy	Normal leaf with no disease

How it learns (simple idea): We show the model thousands of labelled leaf images. At first its guesses are random. After every guess it checks the right answer, measures how wrong it was, and adjusts its internal numbers a tiny bit. After many rounds it becomes good at recognising disease patterns such as brown spots and rings. This is called **supervised learning** because every training image comes with a correct label.

The complete pipeline:

Step	What happens	Where in the code
1	Load images from folders and split 80% / 20%	Section 2 of the script
2	Build a CNN (Convolutional Neural Network)	Section 3
3	Train the CNN on the training images	Section 4
4	Test it on images it has never seen	Section 5
5	Save the model, graphs and results	Section 6
6	Use the saved model in a GUI (gui_app.py)	Separate file

2. Dataset and the 80/20 Split

The dataset has one folder per class. The **folder name is the label**, so we never have to write labels by hand. This is why the function used can read labels directly from the directory structure.

Why split the data? If we tested the model on the same images it learned from, it could simply memorise them and look perfect. That tells us nothing about real use. So we hide some images and only use them at the end, like an exam paper the student has not seen before.

Part	Share	Purpose
Training set	about 72%	The model learns from these images
Validation set	about 8%	Checked after every epoch to watch for overfitting; helps pick the best model
Test set	20%	Used only once at the end for the final, honest accuracy

The script first splits 80% (training) and 20% (testing), as required. It then takes 10% of the 80% as a validation set. That is why the training share is shown as about 72% and validation as about 8%.

3. Libraries Used

Library	Why we use it
os, json	Work with folders/paths; save the class names in a small text file
numpy	Fast maths on arrays of numbers (images are arrays of pixel values)
tensorflow / keras	The deep learning framework: builds, trains and saves the model
matplotlib	Draws the accuracy/loss graphs
seaborn	Draws the coloured confusion matrix
scikit-learn (sklearn)	Gives ready-made evaluation tools: classification report and confusion matrix

4. Code Walk-through, Section by Section

Section 1 - Configuration

```
DATA_DIR = r"C:\Users\Santanu\Desktop\Project\  
    Plant_leave_diseases_dataset_with_augmentation" # one line in the real file  
OUTPUT_DIR = r"C:\Users\Santanu\Desktop\Project_output"  
CLASS_NAMES = ["Potato__Early_blight", "Potato__Late_blight", "Potato__healthy"]  
IMG_SIZE = (128, 128); BATCH_SIZE = 32; EPOCHS = 20; SEED = 42
```

- **DATA_DIR** - where the images are. **OUTPUT_DIR** - where the model and graphs are saved. It is kept outside the dataset folder so it is never mistaken for a class.
- **CLASS_NAMES** - tells Keras exactly which folders to use, ignoring any others.
- **IMG_SIZE (128x128)** - every image is resized to the same size because a neural network needs a fixed input size. Smaller images train faster; larger images keep more detail.
- **BATCH_SIZE (32)** - the model looks at 32 images at a time, then updates itself. This is faster and more stable than one image at a time.
- **EPOCHS (20)** - one epoch is one full pass through all training images. We allow up to 20 passes.
- **SEED (42)** - makes the random shuffle repeatable, so the train/test split is the same every time and never overlaps.

Section 2 - Loading the data

```
train_full = tf.keras.utils.image_dataset_from_directory(  
    DATA_DIR, validation_split=0.20, subset="training", seed=SEED,  
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode="int",  
    class_names=CLASS_NAMES)  
test_ds = tf.keras.utils.image_dataset_from_directory(  
    ..., validation_split=0.20, subset="validation", ...)
```

This one function reads all images, resizes them, attaches labels from folder names, groups them into batches of 32 and splits them. **validation_split=0.20** with **subset="training"** gives the 80% part; **subset="validation"** gives the other 20%, which we use as our **test set**. **label_mode="int"** means labels are numbers: 0 = Early blight, 1 = Late blight, 2 = healthy. Both calls use the same seed so the two parts never overlap.

Validation carved out of training:

```
n_batches = tf.data.experimental.cardinality(train_full).numpy()  
val_batches = max(1, int(0.10 * n_batches))  
val_ds = train_full.take(val_batches) # first 10% of batches  
train_ds = train_full.skip(val_batches) # the rest
```

take grabs the first few batches for validation and **skip** keeps the remaining ones for training.

Speed-up settings

```
train_ds = train_ds.cache().shuffle(1000).prefetch(AUTOTUNE)
```

- **cache()** - keeps images in memory after the first epoch so they are not re-read from disk each time.
- **shuffle(1000)** - mixes the order every epoch so the model does not learn the order of the images.
- **prefetch()** - prepares the next batch while the current batch is being trained, so the processor never waits.

Section 3 - Building the model

Data augmentation creates slightly changed versions of training images on the fly:

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2)])
```

Flipping, rotating and zooming make the model see more variety, so it learns what a diseased leaf looks like in general rather than memorising specific photos. This helps reduce **overfitting**.

Augmentation is only active during training, not when testing.

The full model is covered in detail in Section 5 of this guide (the CNN layer by layer).

Section 4 - Compile and train

```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history = model.fit(train_ds, validation_data=val_ds,
                   epochs=EPOCHS, callbacks=callbacks)
```

Setting	Simple meaning
optimizer = Adam	The method used to adjust the model's internal numbers after each batch. Adam adapts its step size automatically and works well in most cases.
loss = sparse categorical crossentropy	The 'how wrong am I' score for multi-class problems. 'Sparse' means labels are plain numbers (0, 1, 2). Training tries to make this score as small as possible.
metrics = accuracy	The percentage of images classified correctly - easy for humans to read.
model.fit()	Starts the actual training. It returns <i>history</i> , which stores accuracy and loss for every epoch so we can plot graphs.

Callbacks are helpers that run during training:

- **EarlyStopping (patience=5)** - if validation loss does not improve for 5 epochs in a row, stop training and go back to the best weights. This saves time and prevents overfitting.
- **ModelCheckpoint** - saves the model each time validation accuracy reaches a new best, so the best version is never lost.

Section 5 - Evaluation on the test set

```
test_loss, test_acc = model.evaluate(test_ds)
...
print(classification_report(y_true, y_pred, target_names=class_names))
```

The model predicts every test image. We compare predictions (**y_pred**) with the true labels (**y_true**). **np.argmax** picks the class with the highest probability. See Section 6 for how to read the results.

Section 6 - Saving

- **potato_model.keras** - the trained model, loaded later by the GUI.
- **class_names.json** - the class order, so the GUI knows that output 0 means Early Blight, and so on.

- **training_curves.png** and **confusion_matrix.png** - figures for the project report.

5. How the CNN Works, Layer by Layer

A **Convolutional Neural Network (CNN)** is a type of neural network designed for images. Instead of looking at the whole image at once, it slides small filters over it to detect simple patterns (edges, spots, colour changes) and then combines them into complex ones (lesions, rings, leaf shapes). Early layers find simple features; deeper layers find complicated ones.

Layer	What it does (simple words)
Input 128x128x3	A colour image: 128 pixels wide, 128 high, 3 colour channels (red, green, blue).
RandomFlip / Rotation / Zoom	Augmentation (training only).
Rescaling(1/255)	Pixel values are 0-255. Dividing by 255 makes them 0-1, which helps the network learn faster and more stably.
Conv2D (filters, 3x3, ReLU)	The feature detector. Each filter is a small 3x3 window that slides over the image looking for one pattern. More filters = more patterns learned. ReLU keeps positive values and turns negatives into 0, which adds the non-linearity that lets the network learn complex shapes.
MaxPooling2D	Shrinks the image by half by keeping the strongest value in each 2x2 block. Reduces computation and makes the model less sensitive to the exact position of a spot.
Flatten	Turns the final stack of feature maps into one long list of numbers.
Dense(128, ReLU)	A normal fully connected layer that combines all detected features to make a decision.
Dropout(0.5)	During training, randomly switches off half of the neurons each step. This stops the network depending too much on a few neurons and reduces overfitting.
Dense(3, Softmax)	Output layer with one neuron per class. Softmax converts scores into probabilities that add up to 100%. The biggest one is the prediction.

Layer sizes in our model (3x3 filters with no padding shrink each side by 2):

Layer	Output shape	Parameters
Conv2D (32 filters)	126 x 126 x 32	896
MaxPooling	63 x 63 x 32	0
Conv2D (64 filters)	61 x 61 x 64	18,496
MaxPooling	30 x 30 x 64	0
Conv2D (128 filters)	28 x 28 x 128	73,856
MaxPooling	14 x 14 x 128	0
Conv2D (128 filters)	12 x 12 x 128	147,584
MaxPooling	6 x 6 x 128	0

Layer	Output shape	Parameters
Flatten	4,608	0
Dense (128)	128	589,952
Dropout (0.5)	128	0
Dense (3, softmax)	3	387
Total trainable parameters		831,171

Parameters are the numbers the model adjusts while learning. You can confirm these figures in the **model.summary()** output when the script runs. We use filter counts of 32, 64, 128, 128, increasing with depth, because deeper layers need to capture more complex patterns.

6. Reading the Results

Training curves

- **Accuracy curve** should rise and flatten. **Loss curve** should fall.
- If training accuracy is high but validation accuracy is much lower, the model is **overfitting** (memorising).
- If both are low, the model is **underfitting** (not learning enough).

Classification report

Metric	Meaning	Example in our project
Precision	Of all leaves the model labelled as a disease, how many really had it?	Of leaves called Late Blight, how many truly were?
Recall	Of all leaves that truly had the disease, how many did the model find?	Of all real Late Blight leaves, how many did it catch? (Missing a disease is costly.)
F1-score	A single balanced score combining precision and recall.	Higher is better; 1.0 is perfect.
Support	Number of test images in that class.	Shows if classes are balanced.

Confusion matrix

A table where rows are the **actual** class and columns are the **predicted** class. Numbers on the diagonal are correct predictions. Numbers off the diagonal are mistakes, and show exactly which classes get confused (for example Early Blight predicted as Late Blight, since both cause brown lesions).

7. Limitations and Future Scope

Limitations (be honest about these - examiners appreciate it)

- **Augmented dataset:** the dataset name says 'with augmentation', so it may contain altered copies of the same leaf photo. A random split can put a copy in training and the original in testing, which can make test accuracy look higher than real-world accuracy.

- **Lab-style images:** the photos have clean, simple backgrounds. Field photos with soil, shadows or other plants may reduce accuracy.
- **Only 3 classes:** the model must pick one of three, even for a non-potato image. The GUI shows a low-confidence warning to help with this.
- **Image-level only:** it does not locate the infected area on the leaf, nor measure severity.

Future scope

- Use **transfer learning** (MobileNetV2, ResNet) for higher accuracy with less training time.
- Add more crops and diseases, and a 'not a leaf' class.
- Collect real field images to improve robustness.
- Build a mobile app or web app (Streamlit/Flask) for farmers; deploy a light model with TensorFlow Lite.
- Add disease severity estimation and treatment recommendations.

8. Glossary of Key Terms

Term	Simple meaning
Deep learning	Machine learning using neural networks with many layers.
CNN	Neural network specialised for images; uses sliding filters to detect patterns.
Epoch	One full pass through the training data.
Batch	A small group of images processed together (32 here).
Overfitting	Model memorises training data and performs poorly on new data.
Underfitting	Model is too simple or under-trained to learn the pattern.
Data augmentation	Creating varied copies of training images (flip, rotate, zoom) to improve generalisation.
Activation function (ReLU)	A function that lets the network learn non-linear patterns.
Softmax	Converts raw scores into probabilities that sum to 1.
Loss function	Measures how wrong the predictions are; training minimises it.
Optimizer (Adam)	Algorithm that updates the model's parameters to reduce the loss.
Dropout	Randomly switching off neurons during training to prevent overfitting.
Pooling	Shrinking feature maps by keeping the strongest values.
Validation set	Data used during training to monitor progress and tune the model.
Test set	Unseen data used once at the end to measure final performance.
Seed	A fixed starting number that makes random operations repeatable.

9. Questions an Examiner May Ask

Q. Why did you choose a CNN?

A. CNNs are designed for image data. Their filters detect local patterns like spots and edges and share weights across the image, so they need far fewer parameters than a plain fully connected network and work much better on images.

Q. Why split into 80% training and 20% testing?

A. To measure how well the model works on images it has never seen. Testing on training images would hide overfitting. We additionally use a small validation set during training to monitor progress.

Q. What is the difference between validation and test data?

A. Validation data is checked during training to guide decisions such as early stopping and picking the best model. Test data is kept aside and used only once at the end for an unbiased final score.

Q. Why do you resize images to 128x128?

A. The network needs a fixed input size. 128x128 is a balance: small enough to train quickly on a normal PC, large enough to keep the disease spots visible.

Q. Why divide pixel values by 255?

A. Pixels range from 0 to 255. Scaling to 0-1 keeps numbers small, so training is faster and more stable.

Q. What does ReLU do and why use it?

A. ReLU outputs $\max(0, x)$. It adds non-linearity so the network can learn complex patterns, and it is simple, fast and avoids the vanishing-gradient problem of older functions.

Q. What is the purpose of max pooling?

A. It reduces the size of the feature maps, which cuts computation and makes the model tolerant to small shifts in where a feature appears.

Q. How do you prevent overfitting?

A. Four ways: data augmentation, dropout (0.5), early stopping, and monitoring the validation set.

Q. Why softmax in the last layer and not sigmoid?

A. We have three mutually exclusive classes. Softmax gives probabilities across all classes that add up to 100%, so we pick the highest. Sigmoid is for binary or multi-label problems.

Q. Why sparse categorical crossentropy?

A. It is the standard loss for multi-class classification when labels are integers (0, 1, 2). If labels were one-hot vectors we would use plain categorical crossentropy.

Q. What does the Adam optimizer do?

A. It updates the weights after each batch, adapting the learning rate for each parameter, so it usually converges quickly without much tuning.

Q. What is early stopping?

A. It stops training when validation loss stops improving for a set number of epochs (5 here) and restores the best weights, avoiding wasted time and overfitting.

Q. How do you evaluate the model, and why not only accuracy?

A. We use accuracy, precision, recall, F1 and a confusion matrix. Accuracy alone can hide problems, for example if one class is rarely predicted correctly. The confusion matrix shows exactly which classes get mixed up.

Q. What are the limitations of your model?

A. The dataset has clean, lab-style and partly augmented images, so real field accuracy may be lower; it only knows 3 classes; and it does not locate or measure the infection. Future work includes field data and transfer learning.

Q. How can the accuracy be improved?

A. More and more varied real-world images, transfer learning with a pre-trained network like MobileNetV2, tuning hyperparameters (learning rate, filters, dropout), and training longer with early stopping.